

第6章 面向对象程序设计

《Python程序设计，董付国》

面向对象程序设计

- 面向对象程序设计（Object Oriented Programming, OOP）主要针对大型软件设计而提出，使得软件设计更加灵活，能够更好地支持代码复用和设计复用，并且使得代码具有更好的可读性和可扩展性。
- 面向对象程序设计的一条基本原则是计算机程序由多个能够起到子程序作用的单元或对象组合而成，大大地降低了软件开发的难度。
- 面向对象程序设计的一个关键性观念是将数据以及对数据的操作封装在一起，组成一个相互依存、不可分割的整体，即对象（或称实例）。对于相同类型的对象进行分类、抽象后，得出共同的特征和行为而形成类，面向对象程序设计的关键就是如何合理地定义和组织这些类以及类之间的关系。

面向对象程序设计

- Python完全支持了面向对象程序设计的思想，是真正面向对象的高级动态编程语言，完全支持面向对象的基本功能，如封装、继承、多态以及对基类方法的覆盖或重写。
- Python中对象的概念很广泛，Python中的一切内容都可以称为对象，除了数字、字符串、列表、元组、字典、集合、range对象、zip对象等等，函数也是对象，类也是对象。
- 创建类时用变量形式表示的对象属性称为数据成员，用函数形式表示的对象行为称为成员方法，成员属性和成员方法统称为类的成员。

6.1.1 类定义语法

- Python使用**class关键字**来定义类，**class**关键字之后是一个空格，然后是类的名字，再然后是一个冒号，最后换行并定义类的内部实现。
- **类名的首字母一般要大写**，当然也可以按照自己的习惯定义类名，但一般推荐参考惯例来命名，并在整个系统的设计和实现中保持风格一致，这一点对于团队合作尤其重要。
- 定义类时如果不指定基类，则默认为**object**。

```
class Car:  
    def infor(self):  
        print(" This is a car ")
```

6.1.1 类定义语法

- 定义了类之后，可以用来实例化对象，并通过“对象名.成员”的方式来访问其中的数据成员或成员方法。

```
car = Car()  
car.infor()
```

- 可以使用内置函数`isinstance()`来测试一个对象是否为某个类的实例。

```
isinstance(car, Car)          # True  
isinstance(car, str)         # False
```

6.1.1 类定义语法

- Python关键字“**pass**”表示空语句，可以用在类和函数的定义中或者选择结构中。当暂时没有确定如何实现功能，或者为以后的软件升级预留空间，或者其他类型功能时，可以使用该关键字来“占位”。

```
class A:  
    pass
```

```
def demo():  
    pass
```

```
if 5>3:  
    pass
```

6.1.2 self参数

- 类的所有实例方法都**必须至少**有一个名为**self**的参数，并且必须是方法的**第一个**形参（如果有多个形参的话），**self**参数代表将来要创建的对象本身。
- 在类的实例方法中访问实例属性时需要以**self**为前缀。
- 在外部通过**对象**调用对象方法时并**不需要**传递这个参数，如果在外部通过**类**调用对象方法则**需要**显式为**self**参数传递一个该类的对象。

6.1.2 self参数

- 在Python中，在类中定义实例方法时将第一个参数定义为self只是一个习惯，而实际上不必使用self这个名字，尽管如此，建议编写代码时仍以self作为方法的第一个参数名字。

```
>>> class A:
    def __init__(hahaha, v):
        hahaha.value = v
    def show(hahaha):
        print(hahaha.value)
```

```
>>> a = A(3)
>>> a.show()
```

3

6.1.3 类成员与实例成员

- 属于实例的数据成员一般是指在构造方法__init__()中定义的，定义和使用时必须以self作为前缀；属于类的数据成员是在类中所有方法之外定义的。
- 在主程序中（或类的外部），实例属性属于实例(对象)，只能通过对象名访问；类属性属于类，可以通过类名或对象名都可以访问。
- 在实例方法中可以调用该实例的其他方法，也可以访问类属性以及实例属性。

6.1.3 类成员与实例成员

- 在Python中，可以动态地为自定义类和对象增加或删除成员，这一点是和很多面向对象程序设计语言不同的，也是Python动态类型特点的一种重要体现。

6.1.3 类成员与实例成员

```
class Car:
    price = 100000                    # 定义类属性
    def __init__(self, c):           # 定义实例属性
        self.color = c

car1 = Car('Red')                    # 实例化对象
car2 = Car('Blue')
print(car1.color, Car.price)         # 查看实例属性和类属性的值
Car.price = 110000                   # 修改类属性
Car.name = 'QQ'                      # 动态增加类属性
car1.color = 'Yellow'                # 修改实例属性
print(car2.color, Car.price, Car.name)
print(car1.color, Car.price, Car.name)
```

6.1.3 类成员与实例成员

```
import types

def setSpeed(self, s):
    self.speed = s

car1.setSpeed = types.MethodType(setSpeed, car1) # 动态增加成员方法
car1.setSpeed(50)                                # 调用成员方法
print(car1.speed)
```

6.1.3 类成员与实例成员

- Python类型的动态性使得我们可以动态为自定义类及其对象增加新的属性和行为，俗称混入（mixin）机制，这在大型项目开发中会非常方便和实用。
- 例如系统中的所有用户分类非常复杂，不同用户组具有不同的行为和权限，并且可能会经常改变。这时候我们可以独立地定义一些行为，然后根据需要来为不同的用户设置相应的行为能力。

6.1.3 类成员与实例成员

```
>>> import types
>>> class Person(object):
    def __init__(self, name):
        assert isinstance(name, str), 'name must be string'
        self.name = name

>>> def sing(self):
    print(self.name+' can sing.')

>>> def walk(self):
    print(self.name+' can walk.')

>>> def eat(self):
    print(self.name+' can eat.')
```

6.1.3 类成员与实例成员

```
>>> zhang = Person('zhang')
>>> zhang.sing()                                     # 用户不具有该行为
AttributeError: 'Person' object has no attribute 'sing'
>>> zhang.sing = types.MethodType(sing, zhang) # 动态增加一个新行为
>>> zhang.sing()
zhang can sing.
>>> zhang.walk()
AttributeError: 'Person' object has no attribute 'walk'
>>> zhang.walk = types.MethodType(walk, zhang)
>>> zhang.walk()
zhang can walk.
>>> del zhang.walk                                   # 删除用户行为
>>> zhang.walk()
AttributeError: 'Person' object has no attribute 'walk'
```

6.1.3 类成员与实例成员

- 在Python中，**函数和方法是有区别的**。方法一般指与特定实例绑定的函数，通过对象调用方法时，对象本身将被作为第一个参数隐式传递过去，普通函数并不具备这个特点。

6.1.3 类成员与实例成员

```
>>> class Demo:
    pass

>>> t = Demo()
>>> def test(self, v):
    self.value = v

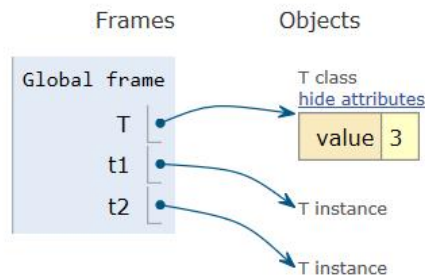
>>> t.test = test
>>> t.test                                # 普通函数
<function test at 0x00000000034B7EA0>
>>> t.test(t, 3)                          # 必须为self参数传值
>>> t.test = types.MethodType(test, t)
>>> t.test                                # 绑定的方法
<bound method test of <__main__.Demo object at 0x000000000074F9E8>>
>>> t.test(5)                            # 不需要为self参数传值
```

6.1.3 类成员与实例成员

- **补充：**在Python中，变量并不直接存储值，而是**存储值的引用**。列表、元组、字典、集合以及其他容器类对象中的元素也是存储值的引用。对象中的成员也是存储的引用。
- 自定义类的数据成员是该类所有对象共有的，既可以通过类访问，也可以通过该类任意对象进行访问。

Write code in Python 3.6 ▼
(drag lower right corner to resize code editor)

```
1 class T:  
2     value = 3  
3  
4 t1 = T()  
5 t2 = T()
```

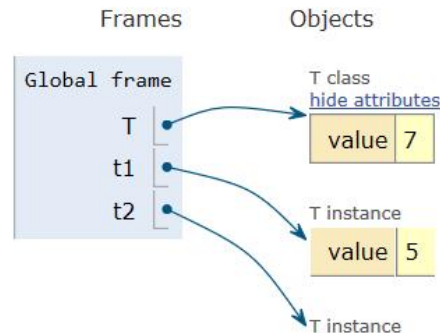


6.1.3 类成员与实例成员

- 如果通过类把成员的值进行了修改，该类对象都能得到体现。然而，如果通过其中某个对象修改了`value`的值，不会影响类和该类其他对象。
- ✓ 修改`t1.value`的值之后，`t1.value`不再共享类的数据成员。
- ✓ 修改`T.value`之后，不影响已改变的`t1.value`，并且`t2.value`仍然共享类的数据成员。

Write code in Python 3.6 ▼
(drag lower right corner to resize code editor)

```
1 class T:  
2     value = 3  
3  
4 t1 = T()  
5 t2 = T()  
6 t1.value = 5  
→ 7 T.value = 7  
8
```



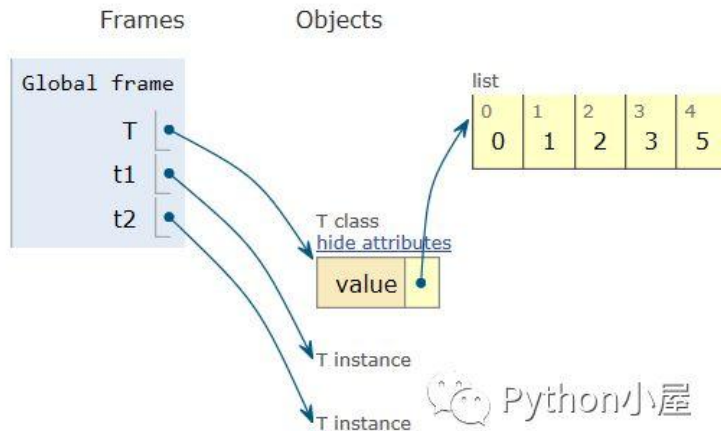
6.1.3 类成员与实例成员

- 当类成员value为列表[1,2,3]时，相应的一系列修改之后，内存布局如图，不管是通过类还是通过该类的对象，只要使用列表自身的原地修改方法或者下标的形式，修改的都是同一个列表。

Write code in Python 3.6

(drag lower right corner to resize code editor)

```
1 class T:
2     value = [1,2,3]
3
4 t1 = T()
5 t2 = T()
6 t1.value.append(4)
7 t2.value.insert(0,0)
8 T.value[-1] = 5
9
```



6.1.3 类成员与实例成员

- 自定义类中的方法也遵守同样的规则。

```
>>> class T:
    def show(self):
        print('T')

>>> def show1(self):
    print('show1')

>>> def show2(self):
    print('show2')

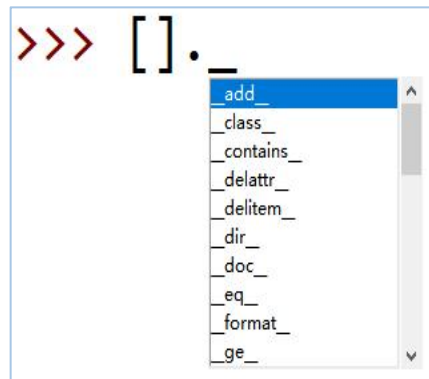
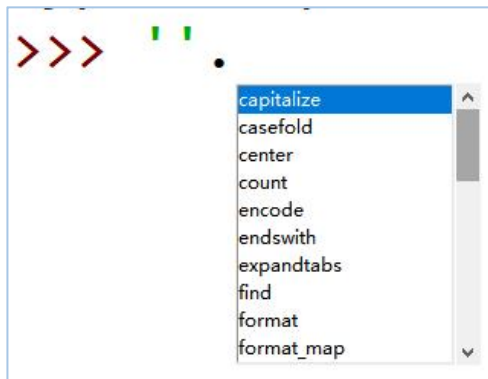
>>> import types
>>> t1 = T()
>>> t2 = T()
>>> t1.show = types.MethodType(show1, t1)
>>> t1.show()
show1
>>> t2.show()
T
>>> T.show = types.MethodType(show2, T)
>>> t1.show()
show1
>>> t2.show()
show2
>>>
```

修改t1.show不影响t2.show

修改T.show会影响t2.show, 但是无法再影响t1.show

6.1.4 私有成员与公有成员

- 在IDLE、Spyder、PyCharm等Python开发环境中，在对象或类名后面加上一个圆点“.”，稍等一秒钟则会自动列出其所有公开成员，模块也具有同样的用法。
- 如果在圆点“.”后面再加一个下划线，则会列出该对象、类或模块的所有成员，包括私有成员。



6.1.4 私有成员与公有成员

- 在Python中，以下划线开头的变量名和方法名有特殊的含义，尤其是在类的定义中。
 - xxx：以单下画线开头受保护成员，默认情况下模块中这类成员无法使用`from module import *`导入；
 - xxx：前后各两个下画线，表示系统预定义的特殊成员，不能随意增加，只能按需进行覆盖；
 - xxx：私有成员，只有类对象自己能访问，子类对象不能直接访问这个成员，但在对象外部可以通过“对象名._类名xxx”这样的特殊方式来访问。
- ❖ 注意：Python中不存在严格意义上的私有成员。

6.1.4 私有成员与公有成员

- 在程序中，下画线可以作为匿名变量来使用，往往用于语法上需要一个变量但实际上并不需要关心该变量的值的场合。

```
div, _ = divmod(2024, 20)          # 只关心整商，不关心余数
for _ in range(3):                 # for循环中不使用循环变量
    print('only a test')
data = [(1,2,3), (1,2,3,4,5,6), (1,2,3,4,5,6,7)]
for a, b, *_ in data:              # 收集第3个以及后面的值但不使用
    print(a, b)
```


6.2 方法

- 在类中定义的方法可以粗略分为两大类：**公有方法**、**私有方法**、**静态方法**、**类方法**和**抽象方法**。
- ✓ 私有方法的名字以两个下画线 “__”开始，每个对象都有自己的公有方法和私有方法，在这两类方法中**可以访问属于类和对象的成员**；
- ✓ 公有方法通过对象名直接调用，**私有方法不能通过对象名直接调用**，只能在属于对象的方法中通过**self**调用或在外通过**Python**支持的特殊方式来调用。
- ✓ 如果通过类名来调用属于对象的公有方法，需要显式为该方法的**self**参数传递一个对象名，用来明确指定访问哪个对象的数据成员。

6.2 方法

- ✓ 静态方法和类方法都可以通过类名和对象名调用，但不能直接访问属于对象的成员，只能访问属于类的成员。
- ✓ 静态方法可以没有参数。
- ✓ 一般将cls作为类方法的第一个参数名称，但也可以使用其他的名字作为参数，并且在调用类方法时不需要为该参数传递值。

6.2 方法

```
>>> class Root:
    __total = 0
    def __init__(self, v):      # 构造方法
        self.__value = v
        Root.__total += 1

    def show(self):             # 普通实例方法
        print('self.__value:', self.__value)
        print('Root.__total:', Root.__total)

    @classmethod                # 修饰器, 声明类方法
    def classShowTotal(cls):    # 类方法
        print(cls.__total)

    @staticmethod              # 修饰器, 声明静态方法
    def staticShowTotal():      # 静态方法
        print(Root.__total)
```

6.2 方法

```
>>> r = Root(3)
>>> r.classShowTotal()           # 通过对象来调用类方法
1
>>> r.staticShowTotal()         # 通过对象来调用静态方法
1
>>> r.show()
self.__value: 3
Root.__total: 1
>>> rr = Root(5)
>>> Root.classShowTotal()       # 通过类名调用类方法
2
>>> Root.staticShowTotal()     # 通过类名调用静态方法
2
```

6.2 方法

```
>>> Root.show()      # 试图通过类名直接调用实例方法，失败
TypeError: unbound method show() must be called with Root instance as
first argument (got nothing instead)

>>> Root.show(r)     # 但是可以通过这种方法来调用方法并访问实例成员
self.__value: 3
Root.__total: 2

>>> Root.show(rr)    # 通过类名调用实例方法时为self参数显式传递对象名
self.__value: 5
Root.__total: 2
```

6.2 方法

- 抽象方法只能在抽象类中定义，抽象类不能实例化，这样做时会报错并提示“`TypeError: Can't instantiate abstract class AbstractBase with abstract method...`”，必须创建抽象类的派生类并实现全部抽象方法之后才能对派生类进行实例化。

6.2 方法

```
import abc

class AbstractBase(abc.ABC):           # 创建抽象类
    def __init__(self, v):
        self.value = v
    def modify(self, v):
        self.value = v
    @abc.abstractmethod
    def show(self):                     # 定义抽象方法，可以没有具体实现
        pass

class Child(AbstractBase):             # 继承抽象类，创建派生类
    def show(self):                    # 实现抽象方法
        print(self.value)

c = Child(3)                           # 实例化，创建派生类的对象
c.modify(5)
c.show()
```

6.3 属性

■ 只读属性

```
>>> class Test:
    def __init__(self, value):
        self.__value = value
```

```
@property
```

```
def value(self):
    return self.__value
```

只读，无法修改和删除

6.3 属性

```
>>> t = Test(3)
```

```
>>> t.value
```

```
3
```

```
>>> t.value = 5
```

```
AttributeError: can't set attribute
```

```
>>> t.v=5
```

```
>>> t.v
```

```
5
```

```
>>> del t.v
```

```
>>> del t.value
```

```
AttributeError: can't delete attribute
```

```
>>> t.value
```

```
3
```

只读属性不允许修改值

动态增加新成员

动态删除成员

试图删除对象属性，失败

6.3 属性

- 可读、可修改、可删除的属性。

```
>>> class Test:
    def __init__(self, value):
        self.__value = value

    def __get(self):
        return self.__value

    def __set(self, v):
        self.__value = v

    def __del(self):
        del self.__value

value = property(__get, __set, __del)
def show(self):
    print(self.__value)
```

6.3 属性

```
>>> t = Test(3)
>>> t.show()
3
>>> t.value
3
>>> t.value = 5
>>> t.show()
5
>>> t.value
5
```

6.3 属性

```
>>> del t.value                # 删除属性
>>> t.value                    # 对应的私有数据成员已删除
AttributeError: 'Test' object has no attribute '_Test__value'
>>> t.show()
AttributeError: 'Test' object has no attribute '_Test__value'
>>> t.value = 1                # 为对象动态增加属性和对应的私有数据成员
>>> t.show()
1
>>> t.value
1
```

6.4 特殊方法与运算符重载--6.4.1 常用特殊方法

- Python类有大量的特殊方法，其中比较常用的是构造方法和析构方法，除此之外，Python还预定义了大量的特殊方法，**运算符重载就是通过重写特殊方法实现的**。
- ✓ Python中类的构造方法是__init__(), 一般用来为数据成员设置初值或进行其他必要的初始化工作，在创建对象时被自动调用和执行。如果用户没有设计构造方法，Python将提供一个默认的构造方法来进行必要的初始化工作。
- ✓ Python中类的析构方法是__del__(), 一般用来释放对象占用的资源，在Python删除对象和收回对象空间时被自动调用和执行。如果用户没有编写析构方法，Python将提供一个默认的析构方法进行必要的清理工作。

6.4.1 常用特殊方法

特殊成员	功能说明
<code>__new__()</code>	类的静态方法，可用于确定是否要创建对象
<code>__init__()</code>	构造方法，创建对象时自动调用
<code>__del__()</code>	析构方法，删除对象时自动调用
<code>__add__()</code>	+
<code>__sub__()</code>	-
<code>__mul__()</code>	*
<code>__truediv__()</code>	/
<code>__floordiv__()</code>	//
<code>__mod__()</code>	%
<code>__pow__()</code>	**
<code>__eq__()</code> 、 <code>__ne__()</code> 、 <code>__lt__()</code> 、 <code>__le__()</code> 、 <code>__gt__()</code> 、 <code>__ge__()</code>	==、!=、<、<=、>、>=
<code>__lshift__()</code> 、 <code>__rshift__()</code>	<<、>>
<code>__and__()</code> 、 <code>__or__()</code> 、 <code>__invert__()</code> 、 <code>__xor__()</code>	&、 、~、^

6.4.1 常用特殊方法

特殊成员	功能说明
<code>__iadd__()</code> 、 <code>__isub__()</code>	<code>+=</code> 、 <code>-=</code> ，很多其他运算符也有与之对应的复合赋值分隔符
<code>__pos__()</code>	一元运算符 <code>+</code> ，正号
<code>__neg__()</code>	一元运算符 <code>-</code> ，负号
<code>__contains__()</code>	与成员测试运算符 <code>in</code> 对应
<code>__radd__()</code> 、 <code>__rsub__()</code>	反射加法、反射减法，一般与普通加法和减法具有相同的功能，但操作数的位置或顺序相反，很多其他运算符也有与之对应的反射运算符
<code>__abs__()</code>	与内置函数 <code>abs()</code> 对应
<code>__bool__()</code>	与内置函数 <code>bool()</code> 对应，要求该方法必须返回 <code>True</code> 或 <code>False</code>
<code>__bytes__()</code>	与内置函数 <code>bytes()</code> 对应，要求必须返回字节串
<code>__complex__()</code>	与内置函数 <code>complex()</code> 对应，要求该方法必须返回复数
<code>__dir__()</code>	与内置函数 <code>dir()</code> 对应
<code>__divmod__()</code>	与内置函数 <code>divmod()</code> 对应
<code>__float__()</code>	与内置函数 <code>float()</code> 对应，要求该方法必须返回实数
<code>__hash__()</code>	与内置函数 <code>hash()</code> 对应，要求必须返回整数，同时实现了特殊方法 <code>__hash__()</code> 和 <code>__eq__()</code> 的类的对象为可哈希对象，这两个方法设置为 <code>None</code> 时为不可哈希对象
<code>__int__()</code>	与内置函数 <code>int()</code> 对应，要求该方法必须返回整数

6.4.1 常用特殊方法

特殊成员	功能说明
<code>__iter__()</code>	与内置函数 <code>iter()</code> 对应，实现了该方法的类的对象为可迭代对象
<code>__len__()</code>	与内置函数 <code>len()</code> 对应，要求必须返回整数
<code>__next__()</code>	与内置函数 <code>next()</code> 对应，同时实现了该方法和 <code>__iter__()</code> 方法的类的对象为迭代器
<code>__reduce__()</code>	提供对 <code>reduce()</code> 函数的支持
<code>__reversed__()</code>	与内置函数 <code>reversed()</code> 对应
<code>__round__()</code>	对内置函数 <code>round()</code> 对应
<code>__str__()</code>	与内置函数 <code>str()</code> 对应，要求该方法必须返回 <code>str</code> 类型的数据
<code>__repr__()</code>	与内置函数 <code>repr()</code> 对应，要求该方法必须返回 <code>str</code> 类型的数据
<code>__getitem__()</code>	按照索引获取值
<code>__setitem__()</code>	按照索引赋值
<code>__delattr__()</code>	删除对象的指定属性
<code>__getattr__()</code>	获取对象指定属性的值，对应成员访问运算符“.”

6.4.1 常用特殊方法

特殊成员	功能说明
<code>__getattr__()</code>	获取对象指定属性的值，如果同时定义了该方法与 <code>__getattr__()</code> ，那么 <code>__getattr__()</code> 将不会被调用，除非在 <code>__getattribute__()</code> 中显式调用 <code>__getattr__()</code> 或者抛出 <code>AttributeError</code> 异常
<code>__setattr__()</code>	设置对象指定属性的值
<code>__base__</code>	该类的基类
<code>__class__</code>	返回对象所属的类
<code>__dict__</code>	对象所包含的属性与值的字典
<code>__subclasses__()</code>	返回该类的所有子类
<code>__call__()</code>	包含该特殊方法的类的对象为可调用对象
<code>__get__()</code>	定义了这三个特殊方法中任何一个的类称作描述符（descriptor），描述符对象一般作为其他类的属性来使用，这三个方法分别在获取属性、修改属性值或删除属性时被调用
<code>__set__()</code>	
<code>__delete__()</code>	

6.4.1 常用特殊方法

- 析构方法调用的时机

```
class Language:
    def __init__(self, name):
        self.name = name
        print(f'对象{self.name}被创建。')

    def __del__(self):
        print(f'对象{self.name}被释放。')
```

```
python = Language('Python')
go = Language('go')
```

```
>>>
```

```
=====
对象Python被创建。
对象go被创建。
```

```
>>>
```

```
E:\Python310>python test.py
对象Python被创建。
对象go被创建。
对象Python被释放。
对象go被释放。
```

6.4.1 常用特殊方法

```
class Language:
    def __init__(self, name):
        self.name = name
        print(f'对象{self.name}被创建。')

    def __del__(self):
        print(f'对象{self.name}被释放。')
```

```
python = Language('Python')
go = Language('go')
del python
del go
```

```
>>>
```

```
=====
对象Python被创建。
对象go被创建。
对象Python被释放。
对象go被释放。
```

```
>>>
```

```
E:\Python310>python test.py
对象Python被创建。
对象go被创建。
对象Python被释放。
对象go被释放。
```

6.4.1 常用特殊方法

```
class Language:
    def __init__(self, name):
        self.name = name
        print(f'对象{self.name}被创建。')

    def __del__(self):
        print(f'对象{self.name}被释放。')

python = Language('Python')
python_2 = python
del python
```

```
>>>
=====
对象Python被创建。
>>>
```

命令提示符

```
E:\Python310>python test.py
对象Python被创建。
对象Python被释放。
E:\Python310>
```

6.4.1 常用特殊方法

```
class Language:
    def __init__(self, name):
        self.name = name
        print(f'对象{self.name}被创建。')

    def __del__(self):
        print(f'对象{self.name}被释放。')

def func():
    python = Language('Python')
    go = Language('go')

func()
```

```
>>>
=====
对象Python被创建。
对象go被创建。
对象Python被释放。
对象go被释放。
>>>
```

6.4.2 案例精选

```
>>> from MyArray import MyArray
>>> x = MyArray(1, 2, 3, 4, 5, 6)
>>> y = MyArray(6, 5, 4, 3, 2, 1)
>>> len(x)
6
>>> x + 5
[6, 7, 8, 9, 10, 11]
>>> x * 3
[3, 6, 9, 12, 15, 18]
>>> x.dot(y)
56
>>> x.append(7)
>>> x
[1, 2, 3, 4, 5, 6, 7]
>>> x.dot(y)
The size must be equal.
>>> x[9] = 8
Index type error or out of range
```

6.4.2 案例精选

```
>>> x / 2
[0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5]
>>> x // 2
[0, 1, 1, 2, 2, 3, 3]
>>> x % 3
[1, 2, 0, 1, 2, 0, 1]
>>> x[2]
3
>>> 'a' in x
False
>>> 3 in x
True
>>> x < y
True
>>> x = MyArray(1, 2, 3, 4, 5, 6)
>>> x + y
[7, 7, 7, 7, 7, 7]
```

6.4.2 案例精选

- **补充例题4** 自定义常量类。
 - ✓ 每个类和对象都有一个叫作__dict__的字典成员，用来记录该类或对象所拥有的属性。当访问对象属性时，首先会尝试在对象属性中查找，如果找不到就到类属性中查找。Python内置类型不支持属性的增加，用户自定义类及其对象一般支持属性和方法的增加与删除。
 - ✓ 在下面定义的常量类中，要求对象的成员必须大写，所有成员的值不能相同，并且不允许修改已有成员的值。

6.4.2 案例精选

```
>>> class Constants:
    def __setattr__(self, name, value):
        assert name not in self.__dict__, 'You can not modify '+name
        assert name.isupper(), 'Constant should be uppercase.'
        assert value not in self.__dict__.values(), 'Value already exists.'
        self.__dict__[name] = value

>>> t = Constants()
>>> t.R = 3
>>> t.R = 4
AssertionError: You can not modify R
>>> t.G = 4
>>> t.g = 4
AssertionError: Constant should be uppercase.
>>> t.B = 4
AssertionError: Value already exists.
```

成员不存在，允许添加
成员已存在，不允许修改

成员必须大写
成员的值不允许相同

6.4.2 案例精选

- **补充例题5** 使用字典模拟稀疏矩阵。

[code\使用字典表示和处理稀疏矩阵.py](#)

class SparseMatrix:

"""使用字典表示和处理二维稀疏矩阵
字典的键表示位置，值表示系数矩阵中该位置的值"""

def __init__(self, lstMatrix=None):

"""把二维列表转换为稀疏矩阵，或创建空矩阵"""

self.__data = {}

if lstMatrix is None:

self.__size = (0, 0)

return

if len(set((len(row) for row in lstMatrix))) != 1:

raise Exception("所有行必须长度一样")

使用字典保存原始矩阵中非0元素的位置和值

for rIndex, row in enumerate(lstMatrix):

for cIndex, value in enumerate(row):

value = float(value)

if value != 0:

self.__data[(rIndex, cIndex)] = value

矩阵尺寸

self.__size = (len(lstMatrix), len(lstMatrix[0]))

def toMatrix(self):

"""转换为矩阵"""

创建全0矩阵

matrix = []

for _ in range(self.__size[0]):

matrix.append([0]*self.__size[1])

修改非0元素

for position, value in self.__data.items():

r, c = position

matrix[r][c] = value

return "["+ "\n ".join(map(str, matrix))+"]"

def __add_sub__(self, anotherSparseMatrix, op):

"""两个矩阵的加法与减法运算通用代码"""

if self.__size != anotherSparseMatrix.__size:

raise Exception("尺寸不一致")

result = SparseMatrix()

result.__size = self.__size

positions = tuple(self.__data.keys())+

tuple(anotherSparseMatrix.__data.keys())

for pos in positions:

result.__data[pos] = eval(str(self.__data.get(pos, 0))+op+

str(anotherSparseMatrix.__data.get(pos, 0)))

return result

def __add__(self, anotherSparseMatrix):

"""两个矩阵相加"""

return self.__add_sub__(anotherSparseMatrix, '+')

def __sub__(self, anotherSparseMatrix):

"""两个矩阵相减"""

return self.__add_sub__(anotherSparseMatrix, '-')

def __mul_div_truediv_pow(self, n, op):

"""矩阵与标量乘法、整除、真除运算通用代码"""

result = SparseMatrix()

result.__size = self.__size

for position, value in self.__data.items():

result.__data[position] = eval(str(value)+op+str(n))

return result

def __mul_matrix(self, anotherMatrix):

"""两个矩阵相乘"""

result = SparseMatrix()

result.__size = (self.__size[0], anotherMatrix.__size[1])

for r in range(self.__size[0]):

获取self矩阵中第r行，列表

row = [0] * self.__size[1]

for pos, value in self.__data.items():

if pos[0] == r:

row[pos[1]] = value

for c in range(anotherMatrix.__size[1]):

获取anotherMatrix矩阵中第c列，列表

col = [0] * anotherMatrix.__size[0]

for pos, value in anotherMatrix.__data.items():

if pos[1] == c:

col[pos[0]] = value

计算两个列表的内积，作为矩阵相乘结果中的一个元素

dot = sum(map(lambda x,y: x*y, row, col))

result.__data[(r,c)] = dot

return result

def __mul__(self, n):

"""乘法，根据参数的不同，调用不同的运算"""

if isinstance(n, (int, float)):

return self.__mul_div_truediv_pow(n, '*')

elif isinstance(n, SparseMatrix) and self.__size[1]==n.__size[0]:

return self.__mul_matrix(n)

else:

raise Exception("数据类型不正确，或尺寸不合适")

def __floordiv__(self, n):

"""矩阵与数字整除"""

assert isinstance(n, int), '除数必须为整数'

return self.__mul_div_truediv_pow(n, '//')

def __truediv__(self, n):

"""真除"""

return self.__mul_div_truediv_pow(n, '/')

def __pow__(self, n):

"""矩阵与数字的幂运算"""

return self.__mul_div_truediv_pow(n, '**')

def __str__(self):

return 'SparseMatrix size:'+str(self.__size)+'\n

'NonZero elements:'+str(len(self.__data))

__repr__ = __str__

6.4.2 案例精选

- 补充例题7 单链表。

[code\单链表.py](#)

```
class Node:
    '''节点结构'''
    def __init__(self, data, nextNode=None):
        # 设置当前节点的值和指向下一个节点的指针
        self.data = data
        self.next = nextNode

    def insertAfter(self, node):
        # 在当前节点后面插入新节点
        node.next = self.next
        self.next = node

    def deleteAfter(self):
        # 删除当前节点后面的节点
        if self.next == None:
            return
        p = self.next
        self.next = p.next
        p.next = None
        del p
```

```
# 头节点
head = Node(0)
p = head
for i in range(1, 10):
    # 依次生成10个数字，并创建相应的节点
    # 把节点连接到链表的尾部
    n = Node(i)
    p.insertAfter(n)
    p = n

p = head
# 遍历链表节点，在值为3的节点后面插入值为3.5的新节点
while True:
    if p.data == 3:
        p.insertAfter(Node(3.5))
        break
    else:
        p = p.next

p = head
# 删除节点7后面的节点
while True:
    if p.data == 7:
        p.deleteAfter()
        break
    p = p.next
    if p.next == None:
        break

p = head
# 遍历链表并输出每个节点的值
while True:
    print(p.data)
    if p.next == None:
        break
    p = p.next
```

6.5 继承机制

- 继承是用来实现**代码复用**和**设计复用**的机制，是面向对象程序设计的重要特性之一。设计一个新类时，如果可以继承一个已有的设计良好的类然后进行二次开发，无疑会大幅度减少开发工作量。
- 在继承关系中，已有的、设计好的类称为**父类或基类**，新设计的类称为**子类或派生类**。派生类可以继承父类的公有成员，但是**不能继承其私有成员**。如果需要在派生类中调用基类的方法，可以使用内置函数**super()**或者通过“**基类名.方法名()**”的方式来实现这一目的。
- Python**支持多继承**，如果父类中有相同的方法名，而在子类中使用时没有指定父类名，则Python解释器将**从左向右**按顺序进行搜索。

6.5 继承机制

■ 构造方法、私有方法以及普通公开方法的继承原理。

```
>>> class A(object):  
    def __init__(self): # 构造方法可能会被派生类继承  
        self.__private()  
        self.public()  
  
    def __private(self): # 私有方法在派生类中不能直接访问  
        print('__private() method in A')  
  
    def public(self): # 公开方法在派生类中可以直接访问，也可以被覆盖  
        print('public() method in A')
```

6.5 继承机制

```
>>> class B(A):          # 类B没有构造方法，会继承基类的构造方法
    def __private(self):  # 这不会覆盖基类的私有方法
        print('__private() method in B')

    def public(self):      # 覆盖了继承自A类的公开方法public
        print('public() method in B')

>>> b = B()              # 自动调用基类构造方法
__private() method in A
public() method in B
>>> dir(b)               # 基类和派生类的私有方法访问方式不一样
['_A__private', '_B__private', '__class__', ...]
```

6.5 继承机制

```
>>> class C(A):
    def __init__(self):      # 显式定义构造方法
        self.__private()   # 这里调用的是类C的私有方法
        self.public()

    def __private(self):
        print('__private() method in C')

    def public(self):
        print('public() method in C')

>>> c = C()                # 调用类C的构造方法
__private() method in C
public() method in C
>>> dir(c)
['_A__private', '_C__private', '__class__', ...]
```

6.5 继承机制

```
class BaseClass(object):
    def show(self):
        print('BaseClass')
```

class SubClassA(BaseClass):

```
    def show(self):
        print('Enter SubClassA')
        super().show()
        print('Exit SubClassA')
```

class SubClassB(BaseClass):

```
    def show(self):
        print('Enter SubClassB')
        #super().show() ← 不再向上一层解析
        print('Exit SubClassB')
```

class SubClassC(SubClassA):

```
    def show(self):
        print('Enter SubClassC')
        super().show()
        print('Exit SubClassC')
```

class SubClassD(SubClassB, SubClassC):

```
    def show(self):
        print('Enter SubClassD')
        super().show()
        print('Exit SubClassD')
```

d = SubClassD()
d.show()
print(SubClassD.mro())

```
Enter SubClassD
Enter SubClassB
Exit SubClassB
Exit SubClassD
[<class, '__main__.SubClassD'>, <class, '__main__.SubClassB'>,
<class, '__main__.SubClassC'>, <class, '__main__.SubClassA'>,
<class, '__main__.BaseClass'>, <class, 'object'>]
```


6.6 多态原理与实现

- 所谓多态（polymorphism），是指基类的同一个方法在不同派生类对象中具有不同的表现和行为。派生类继承了基类行为和属性之后，还会增加某些特定的行为和属性，同时还可能会对继承来的某些行为进行一定的改变，这都是多态的表现形式。
- Python大多数运算符可以作用于多种不同类型的操作数，并且对于不同类型的操作数往往有不同的表现，这本身就是多态，是通过特殊方法与运算符重载实现的。

6.6 多态原理与实现

```
>>> class Animal(object):      # 定义基类
    def show(self):
        print('I am an animal.')
>>> class Cat(Animal):         # 派生类, 覆盖了基类的show()方法
    def show(self):
        print('I am a cat.')
>>> class Dog(Animal):         # 派生类
    def show(self):
        print('I am a dog.')
>>> class Tiger(Animal):       # 派生类
    def show(self):
        print('I am a tiger.')
>>> class Test(Animal):        # 派生类, 没有覆盖基类的show()方法
    pass
```

6.6 多态原理与实现

```
>>> x = [item() for item in (Animal, Cat, Dog, Tiger, Test)]  
>>> for item in x:           # 遍历基类和派生类对象并调用show()方法  
    item.show()
```

```
I am an animal.
```

```
I am a cat.
```

```
I am a dog.
```

```
I am a tiger.
```

```
I am an animal.
```